PRODUCT REQUIREMENTS DOCUMENT

GameLingo

A local-first game translation patch workbench

Fresh build. No paid APIs. No required server. No assumption that old code works.

Core promise

Extract game text, translate it in durable batches, review the result, and export a safe patch without losing hours of work or damaging original files.

Build principle

One well-tested engine profile first. Save editing and live overlays come later, only if the translation workflow earns them.

Start here

This document defines a **clean-slate rebuild**. Treat the current GameLingo repository as historical reference only. No feature, module, test, or compatibility claim from the old project counts as implemented until it is independently rebuilt and verified.

> **The MVP in one sentence**
>
> A Windows desktop app that takes a supported RPG Maker MV/MZ game folder, extracts only known translatable text, translates Japanese to English using a local model, saves progress in SQLite, provides a review editor, and exports or applies a validated patch with automatic backup and restore.

Non-negotiable decisions

- **Cost:** $0 required API spend, backend hosting, account, analytics service, or cloud database. The core app runs locally.
- **Build:** Python-first desktop application, one repo, one process, no web backend in MVP.
- **Translation:** benchmark a small local Japanese-to-English model first; expose local LLM support behind an adapter, not as a hard dependency.
- **Safety:** read-only scan and preview first. Backups and validation before any write. Unknown formats stay read-only.
- **Scope:** no universal engine support, save editor, OCR overlay, mod marketplace, cloud sync, or accounts in MVP.

What success actually looks like

Milestone	Pass condition
First vertical slice	A test RPG Maker MV/MZ project is detected and its known text extracted to structured records without changing source files.
Translation loop	A 1,000-string benchmark runs locally; throughput, cache hits, failures, and human-rated quality are recorded.
Durable job	Pause, force close, reopen, resume, and finish without redoing completed chunks.
Safe output	Preview patch, validate protected tokens, create backup, apply to a test copy, and restore byte-for-byte.
MVP release	A clean install can complete the entire workflow with no API key or hosted service.

Prepared: 10 October 2026. Repository and licensing notes were checked against public pages on that date; re-check before distributing a build. This is a product and engineering plan, not legal advice.

Contents

Non-negotiable decisions . . . 2
What success actually looks like . . . 2

1. Product scope and guardrails . . . 5
1.1 Product definition . . . 5
1.2 Clean-slate assumption . . . 5
1.3 Zero-cost definition . . . 5
1.4 MVP boundaries . . . 5
1.5 Product naming risk . . . 5

2. Users, problem, and product principles . . . 6
2.1 Primary user . . . 6
2.2 Secondary users . . . 6
2.3 Core problems to solve . . . 6
2.4 Product principles . . . 6
2.5 What makes the product worth using . . . 6

3. MVP scope and explicit exclusions . . . 7
3.1 Must ship . . . 7
3.2 Should ship if core remains stable . . . 7
3.3 Explicitly out of MVP . . . 7

4. User workflows and acceptance criteria . . . 8
4.1 First run and project creation . . . 8
4.2 Extract . . . 8
4.3 Translate . . . 8
4.4 Review . . . 8
4.5 Preview, apply, restore . . . 8
4.6 Error states . . . 8

5. UI and information architecture . . . 9
5.1 Navigation . . . 9
5.2 Required screens . . . 9
5.3 Visual direction . . . 9
5.4 UX success rule . . . 9

6. Technical architecture and stack . . . 10
6.1 Recommended free stack . . . 10
6.2 Architecture . . . 10
6.3 Repository layout . . . 10
6.4 Design rules . . . 11

7. Data model and job lifecycle . . . 11
7.1 Core entities . . . 11
7.2 Translation unit identity . . . 11
7.3 Job states . . . 11

7.4 Required persistence behavior . . . 11
7.5 Initial data retention . . . 11

8. Translation performance and quality plan . . . 12
8.1 The first optimization order . . . 12
8.2 Candidate baseline and optional model . . . 12
8.3 Reproducible benchmark . . . 12
8.4 Benchmark gate . . . 12
8.5 Acceptance thresholds for v1 . . . 12

9. Reusable repositories and license decisions . . . 13
9.1 Recommended reuse policy . . . 13
9.2 First-party code licensing . . . 13

10. Safety, privacy, and file integrity . . . 14
10.1 File safety rules . . . 14
10.2 Protected token rules . . . 14
10.3 Local privacy . . . 14
10.4 Input and plugin safety . . . 14
10.5 Boundaries . . . 14

11. Quality, tests, and release gates . . . 15
11.1 Test layers . . . 15
11.2 MVP release gates . . . 15
11.3 Diagnostics . . . 15

12. Non-functional requirements and metrics . . . 16
12.1 Non-functional requirements . . . 16
12.2 Engineering success metrics . . . 16
12.3 Product metrics without telemetry . . . 16

13. Milestone roadmap from an empty repository . . . 17
13.1 Proposed weekly focus, not calendar promises . . . 17

14. First 20 implementation tasks . . . 18
14.1 AI coding assistant rules . . . 18

15. Risks, open decisions, and release checklist . . . 19
15.1 Main risks . . . 19
15.2 Decisions that must be made before public beta . . . 19
15.3 Release checklist . . . 19

16. Free toolchain checklist . . . 20
16.1 Free does not mean effortless . . . 20
16.2 Minimum development workflow . . . 20

17. Source links and verification notes . . . 21
Repositories and models . . . 21
Verification caveats . . . 21

1. Product scope and guardrails

1.1 Product definition

GameLingo is a local-first desktop translation-patch workbench for people who want to translate a game's stored text into a playable, editable translation. It focuses on the slow, repetitive work surrounding large string collections: extraction, de-duplication, translation memory, persistent jobs, terminology consistency, human review, and safe patching.

It is not a live screen translator. It is not a general mod manager. It is not a universal save editor. The first product should make one end-to-end workflow reliable before it tries to make every game compatible.

1.2 Clean-slate assumption

- Create a new repository and architecture. No previous source files are required.
- Do not port old GameLingo modules automatically. Reuse only after comparing behavior, tests, dependencies, and licenses.
- The old repo is not the reference implementation. This PRD is the starting contract; evidence from code and tests overrides guesses.

1.3 Zero-cost definition

Included in $0 target	Not promised
Open-source development tools; local SQLite database; local inference; GitHub repo/issues/releases; local tests.	Electricity, internet bandwidth for initial model download, storage, and the computer itself still have real-world costs.
No required paid APIs, cloud database, server, authentication, telemetry vendor, or subscription.	Optional cloud providers may charge money or change terms. They must be off by default, clearly labeled, and never required.
No promise that unofficial public translation endpoints remain available or may be used indefinitely.	Model downloads take space and need internet once; offline use begins after setup.

1.4 MVP boundaries

> **Definition of done**
>
> A supported test game can be scanned, extracted, translated locally, reviewed, exported, applied to a copy, and restored. Any workflow that requires a paid account, cloud server, hidden API, or destructive write without backup fails the MVP.

1.5 Product naming risk

An unrelated MIT-licensed OCR overlay repository already uses the GameLingo name [R1]. Keep GameLingo as the internal codename for implementation, but decide on a distinct public name before announcing a public release. Name collision affects discovery and may raise trademark/confusion issues; investigate separately.

2. Users, problem, and product principles

2.1 Primary user

Solo player / fan translator. Wants to understand a game that lacks a preferred language. Has the game files but may not know its engine or internal file structure. Needs understandable progress, safe recovery, and no API-key setup.

2.2 Secondary users

- **Technical modder:** wants control over files, terminology, formats, logs, and repeatable builds.
- **Community adapter contributor:** wants a documented adapter interface, deterministic fixtures, and a clear contribution process.

2.3 Core problems to solve

1. Large jobs waste time when duplicate strings are translated repeatedly or completed work is lost after interruption.
2. Translations can damage placeholders, escape sequences, event codes, line structure, or game file syntax.
3. Raw machine output is hard to review consistently across character names, items, terminology, and context.
4. Patching is risky without dry-run preview, backups, checksum verification, and one-click restore.

2.4 Product principles

Principle	Product consequence
Local first	Default path works offline after the model is installed. No telemetry or upload occurs silently.
Resume by design	Translation is a persisted job state machine, not a long-running in-memory loop.
Never guess file structure	Only known fields and commands are translated. Unknown data remains untouched.
Safe by default	Scanning is read-only; patching needs explicit preview and backup.
Explain cost and quality	Show model, language pair, estimated hardware load, cache hit rate, and any online transmission.
Narrow before broad	A small, tested compatibility claim is better than a large unverified engine list.

2.5 What makes the product worth using

Differentiation is not a checkbox count. Several tools already offer game translation, including local models, batching, save editing, and multiple engine profiles [R2, R6]. GameLingo must earn its place through transparent job recovery, precise file preservation, measurable local performance, reviewer-friendly terminology, and trustworthy patch/restore behavior.

3. MVP scope and explicit exclusions

3.1 Must ship

Area	MUST requirement	Acceptance proof
Project	Create/reopen a project stored locally with game path, profile, database, and job history.	Restart app and recover project state.
Detection	Detect RPG Maker MV/MZ only when evidence matches; show confidence and warnings.	Fixtures for positive, negative, and ambiguous folders.
Extraction	Extract known translatable fields and event message commands into units with source location and context.	Golden fixtures and no-source-mutation test.
Translation	Local Japanese-to-English provider, configurable and loaded once per session/job.	Runs without API key once model files exist.
Performance	Exact deduplication, persistent TM, batching, bounded concurrency where useful.	Benchmarks include uncached and warmed-cache runs.
Jobs	Persist progress, retries, pause/cancel/resume, per-chunk checkpoints.	Force-close recovery test passes.
Review	Edit, approve, reject, filter, search, and export translations.	Edits survive app restart.
Safety	Protected-token checks, dry run, backup, atomic write where supported, restore.	Original hash restored exactly in test copy.
Export	Export translated data/patch as well as apply to a copied game folder.	No need to overwrite the original copy.

3.2 Should ship if core remains stable

- User-defined glossary, approved name dictionary, import/export CSV for review, structured diagnostic report, selectable model path, keyboard shortcuts, portable Windows build.
- An optional Ollama-compatible local LLM provider behind a feature flag after the baseline local MT provider is benchmarked.

3.3 Explicitly out of MVP

Deferred item	Why it is deferred	Revisit condition
Universal multi-engine detection	Each engine needs custom parser, writer, tests, edge-case handling, and compatibility maintenance.	First adapter is stable and community demand is evidenced.
Save file editing	Separate schema and corruption risk; does not improve translation workflow.	Stable translation release plus user demand and an isolated design.
OCR / live overlay / hooks	Crowded category and a different real-time problem [R1, R4].	Only as a separate experiment with clear differentiation.
Cloud sync / accounts / analytics	Adds hosting cost, privacy burden, and failure modes.	Not planned unless funding or explicit need changes constraints.
Public plugin marketplace	Untrusted executable code creates security and moderation cost.	A safe adapter manifest and test harness exist first.
Multiplayer, encrypted, anti-cheat or server-side edits	Outside safe single-player translation scope.	Never part of this product.

4. User workflows and acceptance criteria

4.1 First run and project creation

1. User opens GameLingo and sees a short statement that files stay local by default.
2. User chooses a game folder. Detection reads files only and reports engine/profile, matched files, unsupported files, and confidence.
3. User confirms project creation. App creates a separate workspace and stores project metadata; it does not change the game.
4. User can run extraction and inspect a sample before translation.

4.2 Extract

- Every translation unit has a stable ID, source text, file path, JSON/event location, character/speaker context if available, placeholder/control-code inventory, and source checksum.
- Repeated source strings can share a translation candidate, but context-sensitive cases remain distinct when speaker or event context changes quality.
- Extraction is idempotent or versioned. Re-scan shows added, removed, changed, and unchanged units rather than silently destroying review history.

4.3 Translate

- Before calling the model: check translation memory, then exact deduplication, then queue unresolved units.
- Job settings include model, source/target language, batch size, worker count, retry ceiling, timeout, and quality/context mode.
- Progress includes completed/total, errors, cache hits, throughput, elapsed time, ETA estimate, and model/resource status.
- Pause/cancel must stop scheduling new work; already completed chunks remain persisted.

4.4 Review

- Show source, draft, final edit, file/path context, speaker if known, protected tokens, glossary terms, and warnings side by side.
- Statuses: Untranslated, Queued, Draft, Needs Review, Approved, Rejected, Failed, and Stale.
- Search/filter by status, file, warning, source/target content, or glossary term. Manual edits are saved incrementally.

4.5 Preview, apply, restore

- Preview lists changed files, approved units, warnings, validation status, output path, and existing backup state. Dry run performs no writes.
- Default apply target is an exported copy, not the original game folder. Direct apply is an explicit advanced action.
- Before direct writes: verify source checksum, create backup, write to temp path, validate output, then atomically replace where supported.
- Restore can return the backed-up files and verify hashes. Failed writes are logged with recovery instructions.

4.6 Error states

Errors must be actionable and local: unsupported format, permissions, insufficient disk space, missing model, out-of-memory, model output invalid, protected tokens changed, source files changed since extraction, and restore failure. Never show only “something went wrong”.

5. UI and information architecture

5.1 Navigation

```
GameLingo
|-- Home / Game Library
|-- Project Workspace
|   |-- Overview and Compatibility
|   |-- Extraction Results
|   |-- Translation Setup
|   |-- Job Monitor
|   |-- Review Editor
|   |-- Patch Preview and Export
|   |-- Backups and Restore
|   `-- Diagnostics
|-- Translation Memory and Glossary
|-- Models and Providers
`-- Settings and About
```

5.2 Required screens

Screen	Required components	Failure/recovery behavior
Home	Projects, recent job, supported/unsupported state, Add Game.	Empty state has one clear next step; path errors are actionable.
Compatibility	Detected profile, confidence, matched/ignored files, warnings.	Unknown game defaults to read-only scan/inspection.
Extraction	File tree, sample preview, unit count, protected-token report.	Retry failed files; never edit source during extraction.
Translation setup	Model/provider, language pair, glossary, batch/workers, storage estimate.	Missing model offers setup steps, no API-key prompt as default.
Job monitor	Progress, throughput, cache hits, failures, pause/cancel/resume.	Resume uses persisted chunk state; export diagnostics.
Review editor	Source/target, context, flags, glossary, search/filter, approval.	Unsaved changes protected; stale source detected.
Patch preview	Changed files/units, warnings, backup status, output destination.	Apply disabled while blocking validations fail.
Recovery	Backups, timestamps, hashes, restore/dry-run verification.	Restore failure displays exact paths and next steps.

5.3 Visual direction

- Desktop utility, not a marketing website: compact project navigation, a readable data grid, clear progress, and dense-but-legible review tools.
- No neon gradients, no glassmorphism, no huge dashboard cards. Use neutral dark/light themes, restrained teal for safe progress and orange/red for warnings or destructive actions.
- Any destructive operation is named plainly. Warnings cannot depend on color alone.
- Support keyboard navigation, scalable text, sufficient contrast, and sensible focus order.

5.4 UX success rule

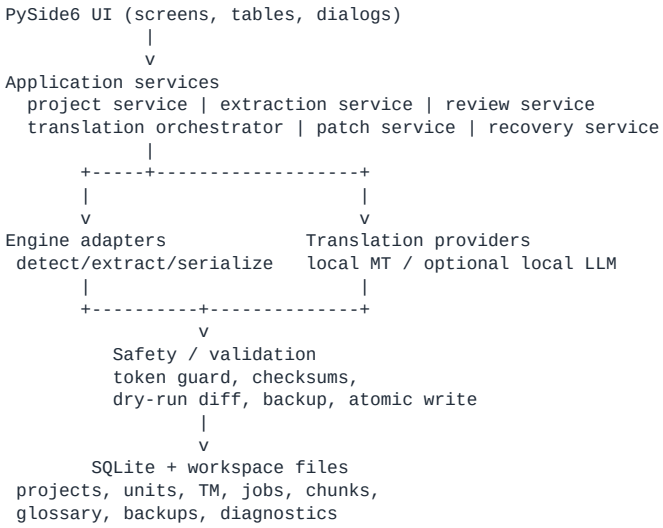
A user should always be able to answer: what is GameLingo doing, what files will change, what data is saved, what remains unfinished, and how do I recover? If the interface hides one of these answers, the flow is incomplete.

6. Technical architecture and stack

6.1 Recommended free stack

Layer	Recommendation	Why	Caveat
Language/runtime	Python 3.12 as initial baseline; lock dependencies.	One language across UI, extraction, jobs, translation, tests.	Re-evaluate only after dependency support is checked.
Desktop UI	PySide6 / Qt for Python.	Strong desktop widgets, tables, model-view patterns, packaging ecosystem.	Community package uses LGPLv3/GPLv3 terms; ship notices and comply with LGPL conditions [R7].
Persistence	Python sqlite3 + SQLite.	Local, serverless database, durable job state; SQLite is public domain [R8].	Use migrations and transactions. Avoid network DB.
Local translation baseline	Helsinki-NLP/opus-mt-ja-en as benchmark candidate.	Dedicated ja-en model; Apache-2.0 model card [R9].	Model quality on game dialogue is not guaranteed; benchmark before defaulting.
Optional local LLM	Ollama-compatible provider adapter.	Can offer richer context without a cloud API.	Model memory, speed, and license vary. Not MVP blocker.
Project/dependency tools	Git, GitHub, uv, pytest, Ruff.	Free workflows, repeatable environments, fast local tests.	Do not treat any hosted free quota as unlimited.
Packaging	PyInstaller portable build after core is stable.	No required Python install for end users.	Audit bundled dependency and model licenses; installer size can be large.

6.2 Architecture



6.3 Repository layout

```
gamelingo/
  pyproject.toml
  README.md  LICENSE  THIRD_PARTY_NOTICES.md
  src/gamelingo/
    app.py
    domain/        models.py, statuses.py
    services/      projects.py, extraction.py, translation.py, patching.py
    adapters/      base.py, rpgmaker_mv_mz.py
    providers/     base.py, opus_mt.py, ollama_local.py (later)
    jobs/          runner.py, checkpoint.py, retry.py
    storage/       database.py, migrations/
    safety/        tokens.py, backup.py, atomic_write.py
    ui/            pages/, components/
  tests/
    unit/ integration/ fixtures/rpgmaker_mv_mz/
  benchmarks/     corpus.csv, benchmark.py, reports/
  docs/           architecture.md, adapter_guide.md, privacy.md
  scripts/        license_audit.py, smoke_test.py
```

6.4 Design rules

Keep UI, adapters, providers, persistence, and safety code separate. Long-running work emits progress events; providers return structured errors and never write files; every write passes through one safety service. Use typed/versioned records and schema migrations, and never unpickle untrusted project data.

7. Data model and job lifecycle

7.1 Core entities

Entity	Minimum fields	Purpose
Project	id, title, install_path, adapter_id, profile_version, created_at	Isolates one game workspace.
GameFile	id, project_id, relative_path, format, source_hash, size, detected_at	Tracks source files and validates stale changes.
TranslationUnit	id, file_id, locator, source_text, context, tokens_json, source_hash, status	One translatable string with exact source location.
TranslationRecord	id, unit_id, provider, model, draft_text, final_text, quality_flags, timestamps	Keeps machine output separate from approved human edits.
TranslationMemory	source_hash, language_pair, context_signature, target_text, origin, approval	Reuses verified translations safely.
Job / Chunk	id, settings_json, status, counts, chunk_ids, attempt, error, timestamps	Persistent restartable work.
GlossaryTerm	project_id, source_term, target_term, notes, case_sensitive, priority	Terminology consistency.
BackupManifest	id, project_id, file_path, original_hash, backup_path, created_at	Restore and audit trail.

7.2 Translation unit identity

Do not key only on source text. At minimum, retain an immutable unit ID and a translation lookup key composed of language pair, normalized source text, and a context signature. Use exact matches first. Broader fuzzy reuse can create plausible but wrong dialogue when speaker, gender, or situation differs.

7.3 Job states

```
CREATED -> QUEUED -> RUNNING -> PAUSED -> RUNNING
                             |          |
                             v          v
                         CANCELLED  COMPLETED
                             |
                             v
                         RESUMABLE (if user chooses resume)

Chunk status: PENDING | RUNNING | SUCCEEDED | RETRYABLE_FAILED | FAILED
A chunk is checkpointed after durable results commit to SQLite.
```

7.4 Required persistence behavior

- Commit each successful chunk in a SQLite transaction. Never hold all successful translations only in memory.
- On startup, reconcile RUNNING jobs and stale RUNNING chunks into resumable states. Do not mark success before data is durable.
- Store attempt count, last error category, provider/model, start/end time, and output checks.
- Never resume blindly if the underlying source file checksum changed. Mark affected units STALE and require re-extraction or an explicit remap.

7.5 Initial data retention

All project databases, translation memory, logs, and backups live in a user-chosen local data directory. Logs must not include full source game text by default. Users can inspect, export, or delete a project and its cache without contacting a server.

8. Translation performance and quality plan

8.1 The first optimization order

1. **Measure first.** Record extraction time, model load time, time per batch, outputs per second, retries, cache hits, and peak memory.
2. **Exact deduplication.** Translate one exact duplicate once when context is equivalent; do not normalize away punctuation or control tokens that affect meaning.
3. **Persistent translation memory.** Reuse prior approved or model-drafted outputs according to quality status and context signature.
4. **Durable batches.** Tune batch size and worker count against measured throughput and memory use; never use unlimited concurrency.
5. **Checkpoint and retry.** A failed batch should be retried independently with bounded exponential backoff and clear failure classification.
6. **Model optimization.** Only after the pipeline is safe, test quantization/conversion such as CTranslate2 or a local LLM provider. Measure whole-job throughput, not demo latency.

8.2 Candidate baseline and optional model

Candidate	Role	How to evaluate
Helsinki-NLP/opus-mt-ja-en [R9]	Required local MT baseline for Japanese -> English.	Measure runtime, peak RAM/VRAM, naturalness, names, honorifics, placeholders, long dialogue, UI text, and consistency. Model card lists Apache-2.0; verify again before bundling.
Ollama-compatible local LLM	Optional later quality mode.	Test one small model and one larger model that actually fit target hardware. Record model version, quantization, peak memory, latency, coherence, and translation consistency.
Optional online providers	Not required and not default.	Only after local path works; show cost/terms/transmission clearly and let user disable completely. Never depend on unofficial public endpoints.

8.3 Reproducible benchmark

Create a fixed corpus of 1,000 strings sampled across dialogue, choices, item names, skills, descriptions, system messages, placeholders, control codes, duplicates, and long lines. Keep a separate 100-string human-rated set with a rubric. Use lawful test content or synthetic fixtures; do not bundle proprietary game files.

Metric	Record it as
Throughput	Strings/minute and non-empty output tokens/second; include model warm-up and cold start separately.
End-to-end time	Extraction + translation + validation + export time for 1,000 strings.
Reuse	Exact duplicate count, translation-memory hit rate, provider/model calls avoided.
Reliability	Failures, retries, malformed outputs, crash recovery, and repeated-work count.
Quality	Human rating for meaning, fluency, terminology, character voice, context, and token preservation.
Resource use	Peak RAM, peak VRAM when available, disk footprint, model download size, CPU/GPU mode.

8.4 Benchmark gate

> **Do not claim “best”, “fastest”, or “Google quality” without evidence.**
>
> Choose the default provider only after running the same corpus on the target machine with at least two viable settings. Publish the result with the model ID/version and hardware. A model that is fast but repeatedly breaks tokens is not a valid default.

8.5 Acceptance thresholds for v1

- 100% of protected-token mismatches block direct patch application.
- 0 completed chunks are lost in forced-exit/resume tests.
- 0 game-file changes occur during scan, extraction, or dry-run preview.
- Performance target is established after baseline; no fabricated speed target before testing.

9. Reusable repositories and license decisions

The goal is to avoid reinventing useful plumbing without importing another project's license, architecture, or maintenance debt. “Public on GitHub” does not automatically mean reusable. Copy only after checking the exact LICENSE file, bundled assets, transitive dependencies, and the commit you use.

Repo / asset	Current signal	Recommended use	Decision
fbscarel/gamelingo [R1]	MIT-licensed OCR overlay; local OCR plus public Google translation wrapper. Shares the GameLingo name.	Read its structure for caching, settings, OCR boundary, and UI ideas. Its live overlay workflow is not the core product.	Reference or isolated later experiment; not MVP foundation.
rpgmaker-translator4 [R2]	Business Source License 1.1 with an expressly non-commercial use grant; states it is not an open-source license.	Study its feature ideas and benchmark behavior; do not copy code into a project that may later be commercial.	Reference only.
DazedMTLTool / DazedTL [R3]	GitHub surfaces an MIT license link, while its README says DazedTL source is available under GPL-3.0. Treat the current source/license relationship as ambiguous until manually resolved.	Useful to study engine workflows, context building, review, patching, and tests. Do not port modules until license, exact commit, assets, and dependencies are audited.	Reference only until license audit.
LunaTranslator [R4]	GPL-3.0; large Windows-oriented visual-novel translator with text hooks, OCR, and many translation providers.	Study provider abstraction, troubleshooting, and user-facing settings. Its hook/overlay feature set is a different product and has native dependencies.	Architecture reference; do not copy casually.
Universal Save Editor [R5]	MIT; TypeScript/browser-based schema-driven save editor with local client-side processing.	Good future reference for plugin/schema-driven save editing after v1. Its TypeScript/browser implementation is not a drop-in Python module.	Phase 3+ research only.
RuneTranslatePublic [R6]	Public repo describes game-file patching, multiple engines, and a save editor; a clear reuse license was not established from the inspected repo page.	Competitive workflow analysis. Compare UX, supported formats, export flow, costs, and limitations.	Reference only unless a clear LICENSE is verified.
OPUS-MT ja-en model [R9]	Model card lists Apache-2.0.	Local Japanese-English benchmark baseline. Keep model files separate from source code; preserve attribution/license notices if redistributing.	Benchmark candidate; verify license and performance.

9.1 Recommended reuse policy

- Borrow ideas freely; copy code only with a confirmed compatible license and retained notices.
- Maintain `THIRD_PARTY_NOTICES.md` with component, version/commit, source URL, license, local modifications, and distribution notes.
- Maintain `MODEL_CARDS.md` with model name/version, license, source, size, download status, and known limitations.
- Do not bundle other games, assets, scripts, save files, copyrighted text, or extracted data in tests without permission.
- If integrating GPL code, deliberately decide whether the distributed derivative should be GPL. Do not make that decision accidentally through copy-paste.

9.2 First-party code licensing

Recommended initial license for clean-room GameLingo code: MIT, provided all included code and assets are compatible. An MIT license does not make third-party models, Qt libraries, bundled assets, or game content automatically MIT. Review the entire distribution set before the first public release.

10. Safety, privacy, and file integrity

10.1 File safety rules

1. Scanning and extraction are read-only. They never rename, overwrite, or delete source files.
2. The default output is a new translated copy or patch folder. Original-directory writes are an explicit advanced path.
3. Create a manifest and file hash before any write. If the source hash changed since extraction, block patching until refreshed.
4. Use a temp file, parse/validate the output, then atomic replace where the filesystem supports it. On failure, retain the original and log recovery instructions.
5. Backups are separate from exported files. Verify a restore by comparing checksums.

10.2 Protected token rules

Extract and record structural tokens before translation, including RPG Maker escape sequences, variables, control codes, placeholders, newline-sensitive separators, and any adapter-specific syntax. Replace sensitive tokens with stable sentinels before model input only where appropriate; verify every required token after translation and restore the original token spelling exactly.

- A mismatch becomes a blocking quality flag, not a soft warning.
- Unknown event commands, script fields, and plugin-specific structures are preserved byte-for-byte or left untranslated.
- No model output is trusted as valid JSON or executable code. Parse and validate it through the adapter.

10.3 Local privacy

- No analytics, tracking, account system, or default network calls.
- Model download is the only required internet interaction after installation; show the source and size before download.
- Optional online translation requires explicit opt-in, clear disclosure of the exact text sent, provider identity, and any known cost.
- Logs default to metadata, exception class, and file path; do not dump all extracted dialogue or save content by default.

10.4 Input and plugin safety

- Reject path traversal and unsafe archive paths. Apply size limits and timeouts when reading archives or malformed files.
- Never execute game JavaScript, arbitrary adapter scripts, downloaded plugins, or untrusted model output.
- Initial adapters are first-party modules. Community format support begins with a declarative manifest plus reviewed Python adapter code; it is not a plugin marketplace.

10.5 Boundaries

The app is for files the user is entitled to modify, with an initial focus on offline single-player fan translation. Do not implement bypasses for DRM, anti-cheat, encrypted server-side progression, online economies, or multiplayer state. The app must respect game licenses and distribution rules.

11. Quality, tests, and release gates

11.1 Test layers

Layer	What is tested	Must-pass examples
Unit	Normalization, token detection, keys, status transitions, retries, checksums.	Token changes are detected; job status cannot jump illegally.
Adapter fixture	Known synthetic RPG Maker MV/MZ files and expected extracted paths.	Extract output matches golden fixture; non-target fields remain identical.
Round-trip	Read -> extract -> patch identity/no-op -> serialize.	No-op patch preserves structural content; supported fields only change when approved.
Persistence	SQLite transaction and restart behavior.	Completed chunks survive abrupt process termination.
Integration	Local model + queue + review + export.	A fixed mini-project completes without network or API key.
Recovery	Backup, overwrite prevention, restore, crash during apply.	Original test-copy files match pre-apply hashes after restore.
Performance	Fixed benchmark corpus, cold/warm runs.	Report time, throughput, errors, cache hits, RAM/VRAM.
Packaging	Clean Windows VM/profile.	Launches without Python/dev tools and without external server.

11.2 MVP release gates

- No known blocking token-preservation bugs in the supported adapter fixture set.
- No source-file mutation occurs during detect, extract, translate, or dry-run preview.
- Forced close/restart/resume works during a translation job without losing completed results.
- A patch can be applied to a copy and restored to exact file hashes.
- Local translation works after model installation without a cloud key or server.
- The clean-install path and all third-party notices are documented.
- At least one human-reviewed benchmark report is published with hardware and model versions.

11.3 Diagnostics

Provide a one-click diagnostic export containing app version, OS version, Python/package versions where appropriate, adapter/profile version, model identifier, counts, timing, hashes, and error messages. Exclude API keys, user home directory details unless needed, and full game text. Show a preview before writing the diagnostic package.

12. Non-functional requirements and metrics

12.1 Non-functional requirements

Category	Requirement
Reliability	Completed work is durable after every committed chunk; app crashes do not lose completed results.
Integrity	No direct write without a backup manifest, validation, and explicit user action.
Offline	Scan, extract, review, TM, local translation, export, patch, and restore work offline after model setup.
Performance	Measure end-to-end performance; model load, extraction, TM, validation, and serialization are visible separately.
Maintainability	Adapters and providers communicate through small typed interfaces; UI is separate from core.
Accessibility	Keyboard access, scalable text, readable contrast, focus indicators, and non-color status signals.
Compatibility	Windows 10/11 x64 first. Other OS support only after packaging and test fixtures exist.
Resource control	Configurable worker count, batch size, model load/unload, cache location, and size warnings.
Distribution	Portable build first; no auto-updater required for MVP. Update behavior cannot overwrite project data.
Observability	Structured local logs, job history, per-chunk error categories, and diagnostic export.

12.2 Engineering success metrics

Metric	Target or method
Crash recovery	100% pass on automated forced-exit scenarios for test jobs.
Protected-token integrity	0 mismatches permitted for automatic patch.
No-op round-trip	No unintended changes outside explicitly translated fields.
Translation throughput	Baseline first; optimize based on reported strings/minute and resource use.
TM correctness	Track exact/context hits separately; do not hide context-mismatch errors in one cache metric.
Support accuracy	A game is called “supported” only when its profile passes its fixture suite on the stated version.

12.3 Product metrics without telemetry

Do not add analytics just to collect vanity numbers. For local dogfooding, export optional anonymous aggregate metrics that users inspect before sharing: completed projects, job completion rate, median strings/minute, failed chunk rate, and restore-test pass rate. The feature is opt-in and not sent automatically.

13. Milestone roadmap from an empty repository

This sequence assumes one developer, free tools, and AI coding assistance with limited quotas. AI can produce boilerplate quickly; it does not remove integration, debugging, file-format research, model evaluation, or user testing. Time estimates are intentionally not treated as commitments.

Milestone	Build	Exit gate
M0. Bootstrap	Create clean repo, license decision, Python 3.12 environment, lockfile, CI, lint, test skeleton, third-party/model notice template.	Clean clone installs and passes a placeholder test. No old code assumed.
M1. Adapter proof	Add synthetic RPG Maker MV/MZ fixtures; detect folder; extract allowlisted text; export JSON/CSV report.	No source mutation, golden tests pass, unsupported structures untouched.
M2. Safety core	SQLite project schema, file hashes, backup manifest, patch preview, restore, no-op round-trip.	Can apply/restore a test copy byte-for-byte before translating anything.
M3. Translation benchmark	Downloadable local OPUS-MT candidate, minimal CLI benchmark, token validator, quality sheet.	Written benchmark compares cold/warm speed, errors, resource use, human-rated sample.
M4. Durable engine	TM, exact dedupe, batches, bounded workers, persisted jobs, retry/backoff, pause/resume.	Forced-exit recovery test passes; completed chunks are not redone.
M5. Usable desktop UI	Project library, extraction preview, provider setup, job monitor, review editor, patch preview.	A new user completes the workflow on a synthetic or permissioned test project.
M6. MVP release	Portable build, docs, notices, logs, diagnostic export, clean-machine smoke test.	No API key/server required; release checklist and known limitations published.
M7. Expand carefully	Add second profile, improved models, glossary import/export, adapter contribution docs.	Each new profile passes the same fixture and recovery gates.
M8. Revisit save editor	Separate product brief; evaluate schema-driven local editor after v1 demand.	Evidence of user demand, threat model, save fixtures, license review, independent tests.

13.1 Proposed weekly focus, not calendar promises

1. First block: setup, model/repo license inventory, adapter fixtures, command-line vertical slice.
2. Second block: safety, patch preview, backup, restore, and round-trip tests.
3. Third block: translation benchmark, local provider, token guard, cache.
4. Fourth block: checkpointed job queue and restart/retry tests.
5. Fifth block: review UI, export, and UX error handling.
6. Sixth block: packaging, documentation, real-game test with permission, bug fixing. If it takes longer, narrow the scope rather than skip tests.

14. First 20 implementation tasks

Create these as small GitHub issues. Close each only when its acceptance test is merged and passes in a clean checkout.

ID	Task	Acceptance criteria
GL-001	Create new repo and environment	README, license, pyproject/lockfile, .gitignore, formatting, and tests work from clean clone.
GL-002	Create CI workflow	Lint and unit tests run on push/PR within free quota; no secrets are required.
GL-003	Create synthetic RPG Maker fixtures	Tiny MV/MZ-like fixtures cover messages, choices, names, descriptions, escape codes, and unknown fields.
GL-004	Define adapter protocol	Typed detection/extraction/validation/patch interfaces with contract tests.
GL-005	Implement detection only	Positive, negative, ambiguous results; read-only filesystem behavior.
GL-006	Implement extraction unit schema	Stable IDs, source location, context, token inventory, source hash.
GL-007	Implement extraction for known fields	Allowlisted data fields and event message commands only.
GL-008	Add golden extraction tests	Expected units and locations match; non-target fields preserved.
GL-009	Create SQLite schema/migrations	Projects, files, units, jobs, chunks, TM, translations, backups.
GL-010	Implement backup manifest/checksum	Backups stored separately with file hash and timestamp.
GL-011	Implement no-op patch round-trip	Can serialize an unmodified test project without unintended changes.
GL-012	Implement dry-run diff	Shows intended file/field changes without writing anything.
GL-013	Implement restore service	Restore test copy and compare hashes to original manifest.
GL-014	Build 1,000-string benchmark corpus	Document source, category distribution, dedupes, protected tokens, and rating rubric.
GL-015	Integrate local ja-en baseline	Model is loaded once, inference is offline after download, errors are structured.
GL-016	Implement token guard	Protected-token mismatch blocks approval/apply automatically.
GL-017	Implement exact dedupe and TM	Measure hits; context-sensitive units are not accidentally merged.
GL-018	Implement persistent job chunks	Force exit and resume without losing committed chunks.
GL-019	Build review/job UI	Progress, edit, approve, search, filter, warning state, pause/cancel/resume.
GL-020	Package and run clean-install QA	Documented portable build works without developer runtime; all notices included.

14.1 AI coding assistant rules

- Ask the coding agent to implement one issue at a time. Do not ask it to implement the whole PRD at once.
- Require it to list files changed, tests added, commands actually run, outputs observed, and known failures.
- Do not accept claims such as “tested” unless the exact test command ran and passed.
- Never allow an AI agent to execute untrusted game scripts, modify the developer's real game folder, or download/extract model weights without explicit approval.
- If a requirement conflicts with the repository, stop and report the conflict instead of silently changing the architecture.

15. Risks, open decisions, and release checklist

15.1 Main risks

Risk	Impact	Mitigation
Local model is too slow or low quality	App is technically offline but frustrating or unusable.	Benchmark early; separate pipeline optimizations from model choice; keep provider interface replaceable.
Model does not fit low-end hardware	Crashes, CPU fallback, excessive latency.	Report model requirements, allow model selection, avoid hardcoding a large LLM as the only provider.
Format parser changes game behavior	Broken events or output.	Known fields only, golden fixtures, no-op round-trip, default export to copy.
Scope creep	No finished release.	No save editor/overlay/marketplace until milestone M6 exits pass.
License contamination	Cannot distribute or commercially use code as intended.	License register, exact-commit checks, third-party notices, avoid BSL/unclear repos for code reuse.
Name confusion	Search/discovery problems or legal friction.	Keep internal codename; research alternative public name before launch [R1, R6].
Free endpoint breaks	Translation workflow unavailable.	Local model is the default; online providers are optional.
Fan-translation distribution rules	User or project may violate game terms/copyright.	Do not ship original game data; document responsible use; keep patches/data separate from game assets.

15.2 Decisions that must be made before public beta

- Confirm whether the public repo will be MIT or another license. Do not copy GPL or BSL code accidentally.
- Confirm final public product name and check existing projects/trademarks.
- Choose benchmark corpus and target machine(s); include the real development laptop as one data point but do not assume all users have a discrete GPU.
- Decide whether PySide6 LGPL compliance and packaging constraints are acceptable; include all Qt notices if distributing binaries.
- Define whether OPUS-MT weights will be downloaded by the user or bundled. User download reduces release size but needs an explicit first-run step.

15.3 Release checklist

- [ ] No required paid API key, hosted backend, database service, or account.
- [ ] New install creates local data directories and can import a synthetic game fixture.
- [ ] Supported game profile has documented limits and a passing fixture suite.
- [ ] Token mismatch blocks patch application.
- [ ] Dry-run produces no writes; apply creates backup; restore returns matching checksums.
- [ ] Forced close/resume and provider failure/retry tests pass.
- [ ] Model and dependency licenses/notices are included.
- [ ] Performance report publishes the machine, model version, and method.
- [ ] README explains privacy, limitations, responsible use, and support workflow.

16. Free toolchain checklist

Purpose	Tool	Use in this project
Editor	Visual Studio Code	Source editing, Python support, debugger, Git integration. Use whichever free editor you already work fastest in.
Version control	Git + GitHub	Branches, issues, PRs, releases, changelog. No server-side app needed.
Environment/deps	Python 3.12 + uv	Reproducible environment and lockfile; pin exact package versions for releases.
UI	PySide6	Native desktop widgets. Follow LGPLv3/GPLv3 obligations for the community distribution [R7].
Storage	SQLite / Python sqlite3	Local project database and reliable job checkpoints [R8].
Translation baseline	OPUS-MT ja-en	Free local model candidate with Apache-2.0 model card [R9].
Optional local LLM	Ollama plus a compatible model	A later provider option; model license, hardware fit, and redistribution rights are checked separately.
Tests	pytest	Unit/integration tests and fixtures; MIT-licensed test framework [R10].
Lint/format	Ruff	Fast lint and formatting; lock the version in dev dependencies.
Packaging	PyInstaller	Portable Windows build. Keep third-party notices and run clean-machine QA.
Docs	Markdown in the repo, GitHub Pages optional	Docs remain versioned with code; do not add paid docs hosting.

16.1 Free does not mean effortless

The toolchain has no required subscription, but model quality and speed still depend on the machine. Model files take disk space and an initial download. Hosted services such as GitHub Actions have usage limits. Keep all critical tests runnable locally, and do not make product correctness depend on a free online quota.

16.2 Minimum development workflow

```
1. Create branch for one issue.
2. Add or update failing test first when possible.
3. Implement the smallest change.
4. Run: ruff check .
5. Run: pytest
6. Run the relevant adapter / recovery integration test.
7. Review diff for file writes, secrets, unlicensed assets, and scope creep.
8. Document actual test output in the pull request.
9. Merge only when local and CI tests pass.
```

17. Source links and verification notes

These are implementation and competitor references, not endorsements. License labels can change and a repository can contain code, assets, and model weights under different terms. Re-check the exact commit and every redistributed asset immediately before using or shipping anything.

Repositories and models

[R1] fbscarel/gamelingo. MIT-licensed real-time OCR/translation project. Relevant for name-collision research, UI and caching ideas, but it solves a different job. Open repository.

[R2] NaniteCrafter/rpgmaker-translator4. Repository README describes multi-engine translation, batching and resume. Its LICENSE file currently states Business Source License 1.1, non-commercial use grant, and says it is not an open-source license. Reference only unless terms are separately clarified. Open repository | LICENSE.

[R3] dazedanon/DazedMTLTool. Legacy game translation tool with engine workflows and tests. GitHub surfaces an MIT license link, while the README states “DazedTL source is available under GPL-3.0.” Treat the license relationship as unresolved and do not copy code until manually verified. Open repository.

[R4] HIllya51/LunaTranslator. GPL-3.0 visual-novel translator with text hooks, OCR and many providers; useful as a competitive/architecture reference but not the proposed file-patching core. Open repository.

[R5] DisLoPik/universal-save-editor. MIT-licensed browser-based schema-driven save editor. Relevant only to a future, separately scoped save-editing phase. Open repository.

[R6] 3nginius/RuneTranslatePublic. Product/repository describes patching game files, multiple engines and a save editor. No clear reuse license was established from the inspected repository page; treat as reference only. Open repository.

[R7] Qt for Python licensing. PySide6 community distribution follows LGPLv3/GPLv3/commercial licensing. Read the official licensing requirements before distributing a bundled app. Official docs | Qt licensing.

[R8] SQLite copyright. The SQLite core is dedicated to the public domain according to the official project page. Official copyright page.

[R9] Helsinki-NLP/opus-mt-ja-en. Japanese-to-English translation model; model card lists Apache-2.0 and Transformers examples. Treat it as a baseline candidate, not a quality guarantee. Model card | Opus-MT code license.

[R10] pytest. Free MIT-licensed testing framework. Open repository.

[R11] uv. Free Python package and project manager available under MIT or Apache-2.0. Open repository.

Verification caveats

- Repository pages and model cards were checked on 10 October 2026. A repository description is not a full code audit.
- This PRD does not claim GameLingo currently supports any engine, model, or workflow. Everything listed as MVP is planned until its tests pass.
- This is not legal advice. License compatibility is a release gate, not something to infer from stars, forks, README claims, or the word “free”.

> **Final recommendation**
>
> Build the translation core and safety layer first. Benchmark local translation before promising speed or quality. Reuse clearly licensed components selectively, and keep BSL or ambiguous-license projects in the reference list rather than inside your codebase. Ship one dependable end-to-end workflow before taking on save editing.

End of PRD. Next actionable milestone: M0 - Bootstrap the clean repository and build the first synthetic RPG Maker MV/MZ extraction fixture.